

GM-csőves detektor

- Nukleáris Elektronika házi feladat-
Tekes János, Tálos Gergő

PROGRAMOZÓI DOKUMENTÁCIÓ

Tálos Gergő

PROGRAM RÉSZLETES SPECIFIKÁCIÓJA

A mikrokontrolleres programnak kezelnie kell egy 0-3.3V pulzusokból álló bemenetet, melyeket meg kell számolnia, hogy egy másodperc alatt mennyi érkezett. Ezt ki kell jeleznie egy SPI-os OLED kijelzőn. Ki kell számítani, hogy a mért érték egy 4-20mA-es kimeneten hány milliamperként jelenne meg, azt pedig két darab 7 szegmenses kijelzőn kell megjelenítenie. Egy DAC-on keresztül ki kell adnia azt a feszültséget, hogy a túldoldalon az illesztett hardver segítségével a 7 szegmenses kijelzőkön mutatott érték jelenjen meg. Az OLED-en megjelenik a beütések száma a „CPS:” felirat után, legfeljebb 5 helyiértékkel, illetve a DAC-on megjelenő feszültség a „DAC V:” felirat után.

Egy adott számláló érték felett egy tranzisztor bekapcsolásával megszólaltat egy szirénát (bekapcsol egy hangszórót vagy lámpát), melynek hangja, tápellátása máshonnan érkezik. Az mikrokontroller USB-n keresztül kapja a tápellátását, más részekről függetlenül.

Az eszköz mindig alaphelyzetből indul, a bekapcsolás utáni módosított beállításokat nem jegyzi meg. Az eszközön engedélyezhető és tiltható a funkciók és paraméterek UART-on keresztül a felhasználói dokumentációnak megfelelően, terminálon keresztül módosíthatók, a kommunikáció 9600 Baud/s, 8 bites adathossz, 1 stopbit, paritás nincs.

Az eszközön engedélyezhető és tiltható funkciók:

- a számláló bemenete,
- a DAC kimenete,
- az oled kijelző,
- a 7 szegmenses kijelzők,
- a sziréna,
- az UART és a
- beérkezett értékből a számítások elvégzése

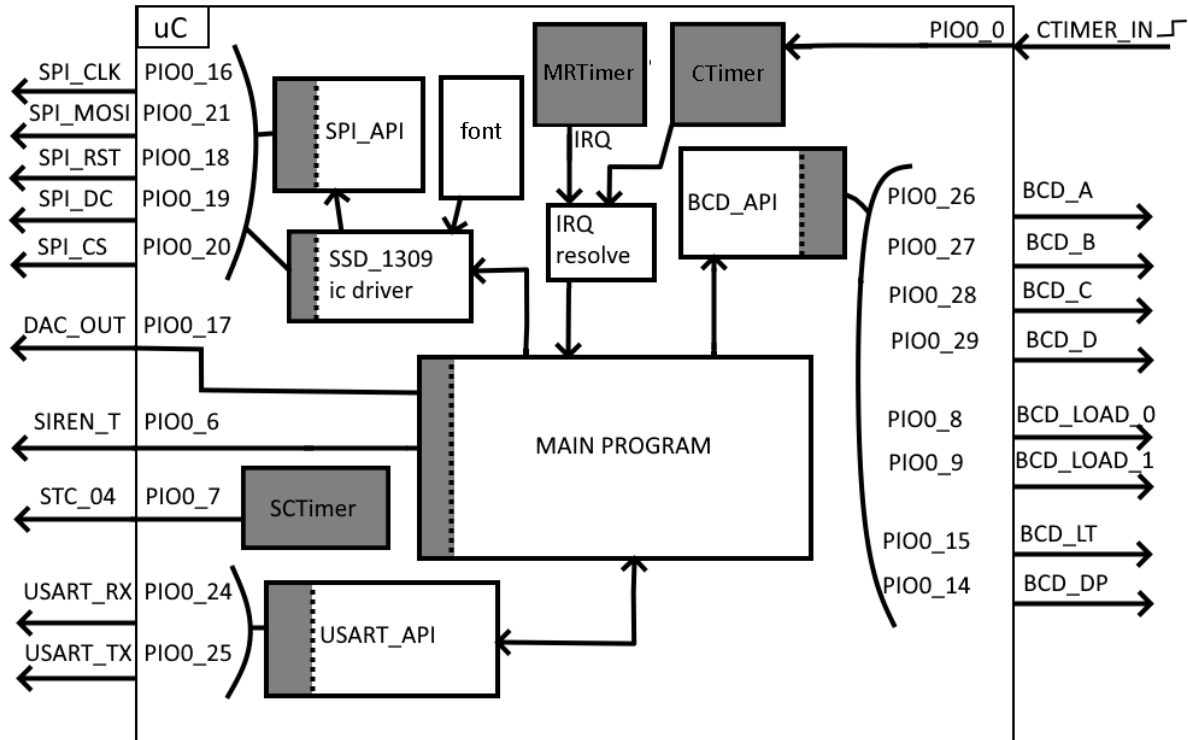
A beállítható paraméterek és alapértelmezett értékeik:

- cps: a bemeneten mért beütésszám, 0;
- min_mA_cps: az alsó mérési határ, mely 4mA-nak felel meg a kimeneten, 0;
- max_mA_cps: a felső mérési határ, mely 4mA-nak felel meg a kimeneten, 1024;
- out_val_R: a DAC periféria kimeneti regiszterébe beírt érték, 0;
- out_val_V: a DAC periférián megjelenő feszültség voltban, 0;
- out_val_mA: a 7 szegmenses kijelzőn megjelenő szám, 4;
- min_offset_D: a DAC periféria mekkora feszültséggel mutasson többet, ahogy az a DAC regiszterében megjelenik, 0;
- min_offset_V: a DAC periféria mekkora feszültséggel mutasson többet, 0;
- siren_cps: a sziréna bekapcsolási határa, mely felett bekapcsol, 60;

A beállított értékeknél USART-on keresztüli módosítást követően a frissítés a következők szerint alakul: Kikapcsolt matematika mellett a cps átírása után nem frissül automatikusan az out_val_R, _V, _mA. A többi paraméter megváltoztatása során a változnak out_val_R, _V, _mA, paraméterek, jelenleg eltárolt cps szerint. A sziréna csak a matematika engedélyezése után kapcsolja az eszköz automatikusan.

A mikrokontroller 30MHz-en működik, a fő ciklus frissítési ideje 10ms. A pontos mérés érdekében 1 darab hardver-megszakítás érkezik a számláló egységtől 1ms elteltével, hogy a következő ciklusban már az új érték kerülhessen kijelzésre, továbbításra.

BLOKKVÁZLAT



Leírás:

A mikrokontroller hardveres részei (GPIO és perifériák vezérlése, maguk a hardver-perifériák) szürke színnel vannak jelölve, a szoftveres részek fehérén maradtak.

A **CTimer** (counter/timer) a **PIO0_0**-s bemenetet figyeli és számol felfelé minden **fel** futó élre. Az **MRT** (multirate timer) másodpercenként megszakítás-kérést generál, melyet a fő programban elhelyezett megszakítás-kezelő függvény végrehajt. Ebben kiolvassa a **CTimer**-ben lévő értéket, azt eltárolja és újraindítja a számolást és az **MRT** **IRQ** periódusát (utóbbi automatikusan is történhet). A program a főciklusban az engedélyezéseknek megfelelően ezt beolvassa, kiszámítja a származtatott értékeket (**DAC** feszültség, 7 szegmensre írandó értékeket) és kiírja azokat. Az **UART**-on beérkezett kéréseket az **USART_API**-n keresztül kezeli, hogyha van ilyen, de ezeket is csak meghatározott időközönként. A kijelzőket egy chip-vezérlőn és két **API**-n keresztül kezeli.

Az **SCTimer** (state configurable timer) egy **PWM** jelet küld ki, hogy az eszköz bemenete tesztelhető legyen önmagával. Ez 500Hz-en megy.

A PROGRAM LEÍRÁSA

A program forrásfájljainak csak azon része kerül bemutatásra, melyet nem a mikrokontroller programozói környezete generált. A máshonnan átemelt részek forrást megjelölve szerepelnek, ezek közül csak a ténylegesen használt függvények kerülnek feltüntetésre.

A fejlesztői környezet által generált kód (órakelek, periféria-beállítások, lábak beállításai és kiosztása) a környezet grafikus felületen, általam került beállításra.

source

apis

BCD_API.h és BCD_API.c

- # define-ok:
 - A BCD-busz, X értéke a lábon megjelenő logikai 1 vagy 0.
 - BCD_A(X), BCD_B(X), BCD_C(X), BCD_D(X) GPIO_PinWrite(...,X)
 - A betöltő lábak. # helyére a helyiérték száma, (külön define sorok minden új helyiérték), X értéke a lábon megjelenő logikai 1 vagy 0.
 - BCD_LOAD_#(X) GPIO_PinWrite(...,X)
 - Egyéb, egyedi lábak
 - BCD_LT(X) GPIO_PinWrite(...,X) // a teszt láb
 - BCD_DP_#(X) GPIO_PinWrite(...,X) // a #. helyiértékű kijelző tizedesvesszője
 - BLANK_CODE 13
 - BCD_DELAY 2 // ~66ns 30MHz-en az 2 ciklus a cpu-n
- uint8_t BCD_digit(uint8_t digit);
A BCD-buszt beállítja a „digit” alsó 4 biten tárolt értékére. Visszatérési érték 0, ha nem érvényes a kód, egyébként 1.
- uint8_t BCD_pint2(uint8_t num);
Két darab, csak pozitív számokat kiírni képes 7 szegmenses kijelzőre kiírja a „num” számot.
- uint8_t BCD_init(void);
Inicializálja a kijelzőt úgy, hogy előbb felvillantja az összes szegmenst, nullázza azokat, vár, majd elsötétíti azokat és ismét vár.
- uint8_t BCD_blank(void);
Elsötétíti az összes kijelzőt.

SPI_API.h és SPI_API.c

- void SPI_MasterStartTransfer(SPI_Type *base, uint8_t data);^[F2]
Az SSD1309 használja, egyszerű SPI protokoll-megvalósítás lekérdezéses (polling) módban.

USART_API.h és USART_API.c

- define-olva vannak a be- és kimeneti tárolók méretei a parancshosszok és maguk a parancsok, ezek a Felhasználói dokumentációban vannak részletezve
- void INIT_USART(void);^[F2]
Inicializálja a körbuffert, ezzel magát az UART-ot.
- uint8_t GetString_TillEndChar(char *str, uint8_t End_Char, uint8_t LenMax, uint8_t reset);^[F2]
Állapotgéppel megoldva, hívásonként egy-egy karaktert olvas be az „str”-be az „End_Char” végjelzésig, maximum „LenMax” mennyiségben. Itt biztosítja a visszatörlést és a loopback-et is, melyet már én tettem bele. Hogyha a reset 1, akkor képes csak új beolvasást kezdeményezni. Visszatérési értéke 1, ha elérte a végjelzést, addig 0.
- void PrintUSART0_NB(char *string);^[F2]
„string” kiírása USART-ra, nem blokkoló módon, hardverrel.

- `double arith(const char *buffer, int8_t i);`
A bemeneti „buffer” tárolóból, „i” offsettel kezd el számot alkotni a bemenetből. Képes negatív és pozitív, egész és tizedes jegyű számokat kezelni. Visszatérési érték az alkotott szám.
- `uint8_t USART_parse(const char* buff, double* set_value, const uint8_t* const logd_in);`
A megadott „buff” tárolóból kiolvassa a parancsot és argumentumait. Amennyiben értéket kell beállítania, azt a „set_value”-ban tárolja el. A `logd_in` ellenőrzi, hogy jogosult-e a hívás más parancsok felismerésére. Visszatérési értéke a felismert parancs kódja:

+ 0 Not logged in	+ 60-68 dis/enable functions
+ 1 login	+ 254 ENTER, empty
+ 2 logout	+ >90, errors
+ 30-38 set parameters	

ic_drivers

fonts.c^[2]

Az SSD1309-es kijelzőhöz készített betűkészlet.

SSD1309.h és SSD1309.c

- `void OLED_init(void);`^[2]
Felprogramozza az oled vezérlőjét. Reseteli, órajelet, eltolást, kezdősort, és orientációt állít be.
- `void OLED_print_string(unsigned char x_pos, unsigned char y_pos, char *ch);`^[2]
A megadott kezdő oszloptól kezd el írni a „ch” szöveget. Az y pozíció karaktersorokban értendő. Hogyha betelt egy sor, a következőben folytatja.
- `void OLED_print_int(unsigned char x_pos, unsigned char y_pos, signed long value);`^[2]
5 jegyű egész számokat képes kiírni a megadott kezdőhelytől a `print_string`-hez hasonlóan.
- `void OLED_print_float(unsigned char x_pos, unsigned char y_pos, float value, unsigned char points);`^[2]
A lebegőpont egész részét a `print_int` hajtra végre, a tizedesvessző utáni rész hossza a „points”-al adható meg, ez 4 jegyig működik, amit a `print_decimal` végez, amiben csak modulusos osztás és karakterkiírás szerepel.
- `void OLED_clear_screen(void);`^[2]
Kitölti üres karakterrel a kijelzőt, vagyis „kiüríti”.

datatypes.h és datatypes.c

- `void nop(unsigned int cyc);`
A processzor „cyc” ciklusig aktívan várakozik, időzítéseknél használatos, például a 7 szegmenses kijelzőnél. Külön fájlban, hogy több fájlból is elérhető legyen.

NukElk_uC_LPC845.c

A program fő alkotóeleme, itt található a fő ciklus, az abban végrehajtandó fő feladatok és az azt megelőző beállítások. Nem minden define és annak értéke kerül megemlíítésre, tekintve, hogy a forráskód is mellékelésre kerül, abban beszédes, további kommentek olvashatók.

- **Fő define-ok:**
 - `SYSTICK SysTick_Config(SystemCoreClock / 1000U);`
A mikrokontrollerbe beépített megszakítás-generáló függvény. Argumentumával beállítható, hogy hány ciklusonként hívódjon meg a `void SysTick_Handler(void)`.
 - USART be-, speciális- és kimeneti tároló hossza
 - Alapértelmezett értékek a paramétereknek
 - SysTick megszakítások beállítása, ami magában foglalja, hogy hány millisecondonként fusson le a fő ciklus és az usart mennyi időnként kerüljön ellenőrzésre.
 - Typedef a paraméterek nevére

• **Volatile változók és alapértékeik/méréteik:**

- `uint32_t vars[10];`
A paraméterek tárolására szolgáló tömb.
- `uint8_t en_reg = 0b01011110`
A funkciókat engedélyező „regiszter”.

* 0x01: cps mérés	* 0x10: sziréna kapcsolója
* 0x02: 4-20 mA kimenet	* 0x20: nem használt (NIX)
* 0x04: Oled kijelző	* 0x40: matek ki és bekapcs
* 0x08: 7 szegmenses kijelző	* 0x80: USART bejelentkezés
- `uint8_t err = 0;`
Hibák tárolására szolgál, főként a későbbi továbbfejlesztésben lenne nagy haszna.
- `uint8_t signal = 0;`
Szemafor a `vars[cps]` (vagyis a beütések számának) kiolvasásához, hogyha foglalt, nem szabad kiolvasni. A beíró függvény mindig megkapja viszont az írási jogot.

• **Függvények**

- `void MRT0_IRQHANDLER(void);`
A MultiRate Timer kezelője, mely pontosan 1 másodpercenként – ha engedélyezve van neki – meghívja a `read_cps(...)` függvényt, miután a megállította CTimert. Felvillant vagy éppen kiolt egy ledet is, hogy lássuk a mérési időt, majd visszaállítja a CTimert és elindítja azt.
- `uint8_t set_vars(volatile uint32_t *vars, double dvalue, Vars_name name);`
Beállítja a „vars” listában a „name” paramétert „dvalue”-ra, megfelelő átalakításokkal; esetleg a hozzá tartozó többbit is. (Pl. az `out_value_reg` magával vonja az `out_value_mA`-t is.) Visszatérési értéke 1, ha hiba keletkezett.
- `uint8_t check_vars(volatile uint32_t *vars, double *dvalue, Vars_name name);`
A „dvalue” értéket vizsgálja meg, hogy megfelel-e a „name” által beállítani kívánt paraméter megköötéseinek. A „vars” paraméterlistát is használja az ellenőrzésekhez. Visszatérési értéke a változó érvényessége. 0, ha érvényes, egyébként 1.
- `void meth_out_value(volatile uint32_t *vars, double out_val);`
Matematikai függvény. A megkapott „out_val” érték (a beütések száma) alapján kiszámolja a többi paramétert. Kiszámítja a DAC-on megjelenő feszültséget, annak regiszterébe beírandó értéket és a túloldalon és a 7 szegmenses kijelzőn megjelenő mA értéket. Hogyha meghaladja a `vars[cps]` a `vars[siren_cps]`-t, akkor bekapcsolja a szirénát. Itt nincsenek lekorlátozva semmiben az értékek, USART-on megjelennek mindenképpen valójukban.
- `void write_ma(volatile uint32_t *vars);`
Beállítja a DAC regiszterét a `vars[out_value_reg]` értékre, hogyha az nem haladja meg annak felső határát, ami 1023. Ebből lesz a feszültség
- `void read_cps(volatile uint32_t *cps);`
Kiolvassa a CTimer tárolójából a beütések számát.
- `void display_texts(char* banner, char* cps_string, char* dac_string);`
Kírja az oledre a fix szövegeket, melyeket ritkán kell frissíteni.
- `int main(void);`
A főprogram és a fő ciklus nagyvonalakban való bemutatása.
 - Hardver inicializálása (órakelek, lábak, perifériák, SysTick)
 - 7 Szegmenses kijelző inicializálása
 - Oled inicializálása, szövegek kiírása
 - USART szövegek és tárolók (bufferek) inicializálása
 - Alapértelmezett értékek beállítása a „vars”-ban
 - MRTimer elindítása.

- Fő, végtelen ciklus. Csak akkor fut le benne valami, hogyha a főciklus idejének számlálója a SysTick_Handler által elérte a 0-t.
 - Engedélyezett funkciók végrehajtása, ha nincsenek, akkor tiltása.
 - CPS kiolvasása a „vars”-ból, hogyha a szemafor inaktív, Matematika, a meth_out_value(...) fv. meghívása.
 - DAC, a write_mA(..) fv meghívása.
 - OLED-re a cps és DAC feszültség kiírása
 - BCD kiírása
 - Sziréna megszólaltatása
 - USART beolvasása, ha lejárt annak számlálója
 - Karakter beolvasása, hogyha végkarakter érkezett:
 - USART_parse(...) meghívása, kód visszatérítése
 - Kód alapján a speciális üzenet kiírása; esetleg paraméter ellenőrzése, majd beállítása; esetleg funkció engedélyezése, tiltása
 - USART kiírása, ha engedélyezett

FORRÁSOK:

- **[F1]** Az eszközhöz publikált használati utasítások és adatlapok, melyek mellékelve lettek.
 - TIL308 adatlap (7 szegmenses kijelző)
 - SSD1309 adatlap (oled panel)
 - SN74192N adatlap (BCD dekóder, nem használt az eszközben)
 - LPC84x.pdf
 - LPC845-BRK-BD2-497116241.png
 - UM1109.pdf
 - UM1181.pdf
 - LPC845-BRK-doc.pdf
- **[F2]** Dr. Benesóczky Zoltán- Mikrokontrollerek alkalmazástechnikája tárgy keretein belül közzétett kódrészletek.
<https://home.mit.bme.hu/~benes/oktatas/vimm9151/JEGYZET/jegyzet.html>
<https://home.mit.bme.hu/~benes/oktatas/vimm9151/JEGYZET/LPC845-BRK/LPC845-BRK.zip>